CSL7110_PROJECT:

DISTRIBUTED STREAMING SYSTEM FOR TWITTER SENTIMENT

Kakarla Sai Swaroop - M25DE1023 - m25de1023@iitj.ac.in

Mugundhan B - M25DE1037 - m25de1037@iitj.ac.in

Abhinav Tote - M25DE1041 - m25de1041@iitj.ac.in

GitHub Repo Link:

https://github.com/Mugundh97B/Twitter-Sentiment-Analysis-Stock.git

Twitter Sentiment Analysis System using Kafka, Spark, and MongoDB

1. Introduction

This project focuses on building a **real-time Twitter sentiment analysis pipeline** using distributed systems and big data technologies. The system ingests streaming data, processes it using machine learning techniques, stores the results, and visualizes them through an interactive dashboard.

Unlike traditional batch processing systems, this project emphasizes **real-time streaming architecture**, enabling continuous ingestion, processing, and visualization of sentiment data.

2. Objective

The main objective of this project is to design and implement a **real-time data pipeline** that:

- Streams tweet-like data continuously
- Performs sentiment analysis using multiple models
- Stores processed results efficiently
- Displays insights through a live dashboard

Additionally, the project evaluates multiple models and compares their performance in terms of **latency and prediction quality**.

3. System Architecture

The system follows a **distributed streaming architecture**, consisting of multiple components working together:

Producer → Kafka → Spark Streaming → MongoDB → Streamlit Dashboard

Components Overview

1. **Kafka Producer**
    - Simulates or streams tweet data
    - Sends messages to Kafka topic (twitter_topic)
2. **Apache Kafka**
    - Acts as a distributed message broker
    - Handles real-time data streaming
3. **Apache Spark Streaming**
    - Consumes data from Kafka
    - Applies sentiment analysis models
    - Computes latency and predictions

4. **MongoDB**
    - Stores processed results
    - Acts as a real-time database
5. **Streamlit Dashboard**
    - Fetches data from MongoDB
    - Displays sentiment insights and analytics

4. Project Structure

The project is organized into modular components as shown below :

- kafka/ → Kafka producer scripts
- spark/ → Streaming and processing logic
- ml/ → Trained machine learning models
- evaluation/ → Model evaluation metrics
- dashboard/ → Streamlit application
- images/ → Screenshots for report
- run.sh → Script to start full pipeline

This modular design ensures maintainability and scalability.

5. Workflow Explanation

The complete workflow of the system is as follows:

Step 1: Data Generation

The Kafka producer generates tweet-like messages containing:

- User
- Text
- Timestamp

These messages are pushed into a Kafka topic.

```
mugulinux@MUGUNDHAN: /mn
pkill -f zookeeper
(twitter-stream) mugulinux@MUGUNDHAN:/mnt/d/MLDB_project/stock$ rm -rf /tmp/kafka-logs
rm -rf /tmp/zookeeper
(twitter-stream) mugulinux@MUGUNDHAN:/mnt/d/MLDB_project/stock$ chmod +x run.sh
chmod: cannot access 'run.sh': No such file or directory
(twitter-stream) mugulinux@MUGUNDHAN:/mnt/d/MLDB_project/stock$ cd ..
(twitter-stream) mugulinux@MUGUNDHAN:/mnt/d/MLDB_project$ chmod +x run.sh
(twitter-stream) mugulinux@MUGUNDHAN:/mnt/d/MLDB_project$ pkill -f kafka
pkill -f spark
pkill -f producer
pkill -f streamlit
pkill -f zookeeper
(twitter-stream) mugulinux@MUGUNDHAN:/mnt/d/MLDB_project$ rm -rf /tmp/kafka-logs
rm -rf /tmp/zookeeper
(twitter-stream) mugulinux@MUGUNDHAN:/mnt/d/MLDB_project$ ./run.sh
 Starting Full Pipeline...
 Using Java:
openjdk version "11.0.30" 2026-01-20
OpenJDK Runtime Environment (build 11.0.30+7-post-Ubuntu-1ubuntu124.04)
OpenJDK 64-Bit Server VM (build 11.0.30+7-post-Ubuntu-1ubuntu124.04, mixed mode, sharing)
 Activating Conda Environment...
 Starting MongoDB...
[sudo] password for mugulinux:
 Starting Zookeeper...
 Starting Kafka Broker...
 Creating Kafka Topic...
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use eit
her, but not both.
Created topic twitter_topic.
 Starting Producer...
 Starting Spark...
 Starting Dashboard...

 ALL SERVICES STARTED (BACKGROUND MODE)
 Open Dashboard: http://localhost:8501

 Logs:
tail -f zookeeper.log
tail -f kafka.log
tail -f producer.log
tail -f spark.log
tail -f dashboard.log
(twitter-stream) mugulinux@MUGUNDHAN:/mnt/d/MLDB_project$
```

Step 2: Streaming via Kafka

Kafka acts as a buffer and distributes messages to consumers.

```
mugulinux@MUGUNDHAN: /mn
OpenJDK 64-Bit Server VM (build 11.0.30+7-post-Ubuntu-1ubuntu124.04, mixed mode, sharing)
📦 Activating Conda Environment...
🍃 Starting MongoDB...
🧠 Starting Zookeeper...
🧠 Starting Kafka Broker...
🎉 Creating Kafka Topic...
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use eit
her, but not both.
Created topic twitter_topic.
📤 Starting Producer...
⚡ Starting Spark...
📊 Starting Dashboard...

✅ ALL SERVICES STARTED (BACKGROUND MODE)
🌐 Open Dashboard: http://localhost:8501

📄 Logs:
tail -f zookeeper.log
tail -f kafka.log
tail -f producer.log
tail -f spark.log
tail -f dashboard.log
(base) mugulinux@MUGUNDHAN:/mnt/d/MLDB_project$ tail -f kafka.log
[2026-04-17 19:01:48,239] INFO [Controller id=0, targetBrokerId=0] Client requested connection close from node 0 (org.apache.kafka.clients.Netwo
rkClient)
[2026-04-17 19:01:48,239] INFO [KafkaServer id=0] started (kafka.server.KafkaServer)
[2026-04-17 19:01:48,413] INFO [zk-broker-0-to-controller-alter-partition-channel-manager]: Recorded new controller, from now on will use node M
UGUNDHAN.:9092 (id: 0 rack: null) (kafka.server.BrokerToControllerRequestThread)
[2026-04-17 19:01:48,452] INFO [zk-broker-0-to-controller-forwarding-channel-manager]: Recorded new controller, from now on will use node MUGUND
HAN.:9092 (id: 0 rack: null) (kafka.server.BrokerToControllerRequestThread)
[2026-04-17 19:01:57,249] INFO Creating topic twitter_topic with configuration {} and initial partition assignment HashMap(0 -> ArrayBuffer(0))
(kafka.zk.AdminZkClient)
[2026-04-17 19:01:57,338] INFO [ReplicaFetcherManager on broker 0] Removed fetcher for partitions Set(twitter_topic-0) (kafka.server.ReplicaFetc
herManager)
[2026-04-17 19:01:57,380] INFO [LogLoader partition=twitter_topic-0, dir=/tmp/kafka-logs] Loading producer state till offset 0 with message form
at version 2 (kafka.log.UnifiedLog$)
[2026-04-17 19:01:57,391] INFO Created log for partition twitter_topic-0 in /tmp/kafka-logs/twitter_topic-0 with properties {} (kafka.log.LogMan
ager)
[2026-04-17 19:01:57,393] INFO [Partition twitter_topic-0 broker=0] No checkpointed highwatermark is found for partition twitter_topic-0 (kafka.
cluster.Partition)
[2026-04-17 19:01:57,395] INFO [Partition twitter_topic-0 broker=0] Log loaded for partition twitter_topic-0 with initial high watermark 0 (kafk
a.cluster.Partition)
```

Step 3: Real-Time Processing using Spark

Spark Streaming reads data from Kafka and performs:

- JSON parsing
- Sentiment analysis using multiple models
- Latency calculation

The processed results are written to MongoDB.

```
mugulinux@MUGUNDHAN: /mn
HAN.:9092 (id: 0 rack: null) (kafka.server.BrokerToControllerRequestThread)
[2026-04-17 19:01:57,249] INFO Creating topic twitter_topic with configuration {} and initial partition assignment HashMap(0 -> ArrayBuffer(0))
(kafka.zk.AdminZkClient)
[2026-04-17 19:01:57,338] INFO [ReplicaFetcherManager on broker 0] Removed fetcher for partitions Set(twitter_topic-0) (kafka.server.ReplicaFetc
herManager)
[2026-04-17 19:01:57,380] INFO [LogLoader partition=twitter_topic-0, dir=/tmp/kafka-logs] Loading producer state till offset 0 with message form
at version 2 (kafka.log.UnifiedLog$)
[2026-04-17 19:01:57,391] INFO Created log for partition twitter_topic-0 in /tmp/kafka-logs/twitter_topic-0 with properties {} (kafka.log.LogMan
ager)
[2026-04-17 19:01:57,393] INFO [Partition twitter_topic-0 broker=0] No checkpointed highwatermark is found for partition twitter_topic-0 (kafka.
cluster.Partition)
[2026-04-17 19:01:57,395] INFO [Partition twitter_topic-0 broker=0] Log loaded for partition twitter_topic-0 with initial high watermark 0 (kafk
a.cluster.Partition)
^C
(base) mugulinux@MUGUNDHAN:/mnt/d/MLDB_project$ tail -f producer.log
Sent: {'timestamp': 1776452592.7701433, 'user': 'user_110', 'text': 'Loss everywhere in market'}
Sent: {'timestamp': 1776452592.8707354, 'user': 'user_88', 'text': 'Bitcoin is falling badly'}
Sent: {'timestamp': 1776452592.9711862, 'user': 'user_419', 'text': 'I hate this market crash'}
Sent: {'timestamp': 1776452593.0716503, 'user': 'user_537', 'text': 'Loss everywhere in market'}
Sent: {'timestamp': 1776452593.1721635, 'user': 'user_177', 'text': 'Investors are happy'}
Sent: {'timestamp': 1776452593.2727373, 'user': 'user_868', 'text': 'Loss everywhere in market'}
Sent: {'timestamp': 1776452593.3732703, 'user': 'user_245', 'text': 'Profit is going high!'}
Sent: {'timestamp': 1776452593.473758, 'user': 'user_789', 'text': 'Tesla stock looks promising'}
Sent: {'timestamp': 1776452593.5743496, 'user': 'user_344', 'text': 'Bitcoin is falling badly'}
Sent: {'timestamp': 1776452593.6749682, 'user': 'user_303', 'text': 'Tesla stock looks promising'}
Sent: {'timestamp': 1776452593.7755363, 'user': 'user_57', 'text': 'I hate this market crash'}
Sent: {'timestamp': 1776452593.8766997, 'user': 'user_957', 'text': 'This is the worst investment ever'}
Sent: {'timestamp': 1776452593.9772055, 'user': 'user_324', 'text': 'This is the worst investment ever'}
Sent: {'timestamp': 1776452594.0776935, 'user': 'user_397', 'text': 'Amazing growth in tech stocks'}
Sent: {'timestamp': 1776452594.1783614, 'user': 'user_444', 'text': 'I hate this market crash'}
Sent: {'timestamp': 1776452594.2792218, 'user': 'user_57', 'text': 'Loss everywhere in market'}
Sent: {'timestamp': 1776452594.3797078, 'user': 'user_710', 'text': 'This is the worst investment ever'}
Sent: {'timestamp': 1776452594.4803271, 'user': 'user_778', 'text': 'Amazing growth in tech stocks'}
Sent: {'timestamp': 1776452594.5814772, 'user': 'user_464', 'text': 'Bitcoin is falling badly'}
Sent: {'timestamp': 1776452594.6819787, 'user': 'user_424', 'text': 'I hate this market crash'}
Sent: {'timestamp': 1776452594.7824302, 'user': 'user_786', 'text': 'Market uncertainty is rising'}
Sent: {'timestamp': 1776452594.8829737, 'user': 'user_849', 'text': 'Investors are happy'}
Sent: {'timestamp': 1776452594.983514, 'user': 'user_193', 'text': 'This is the worst investment ever'}
Sent: {'timestamp': 1776452595.084053, 'user': 'user_321', 'text': 'Bitcoin is falling badly'}
Sent: {'timestamp': 1776452595.1845214, 'user': 'user_852', 'text': 'I hate this market crash'}
Sent: {'timestamp': 1776452595.2852085, 'user': 'user_182', 'text': 'I hate this market crash'}
Sent: {'timestamp': 1776452595.3859613, 'user': 'user_902', 'text': 'Stock market is booming today!'}
Sent: {'timestamp': 1776452595.4866815, 'user': 'user_23', 'text': 'Investors are happy'}
```

```
mugulinux@MUGUNDHAN: /mn
Sent: {'timestamp': 1776452617.31661, 'user': 'user_502', 'text': 'Profit is going high!'}
Sent: {'timestamp': 1776452617.4172065, 'user': 'user_929', 'text': 'Investors are happy'}
Sent: {'timestamp': 1776452617.5178084, 'user': 'user_192', 'text': 'Loss everywhere in market'}
Sent: {'timestamp': 1776452617.6184182, 'user': 'user_976', 'text': 'Stock market is booming today!'}
Sent: {'timestamp': 1776452617.7191646, 'user': 'user_704', 'text': 'Profit is going high!'}
Sent: {'timestamp': 1776452617.8199844, 'user': 'user_896', 'text': 'I hate this market crash'}
Sent: {'timestamp': 1776452617.920533, 'user': 'user_637', 'text': 'This is the worst investment ever'}
Sent: {'timestamp': 1776452618.0211363, 'user': 'user_866', 'text': 'This is the worst investment ever'}
Sent: {'timestamp': 1776452618.1219037, 'user': 'user_280', 'text': 'Amazing growth in tech stocks'}
Sent: {'timestamp': 1776452618.2225356, 'user': 'user_413', 'text': 'Amazing growth in tech stocks'}
Sent: {'timestamp': 1776452618.3231957, 'user': 'user_706', 'text': 'Bitcoin is falling badly'}
Sent: {'timestamp': 1776452618.4238167, 'user': 'user_1', 'text': 'I hate this market crash'}
Sent: {'timestamp': 1776452618.524331, 'user': 'user_587', 'text': 'Bitcoin is falling badly'}
Sent: {'timestamp': 1776452618.6248488, 'user': 'user_63', 'text': 'Stock market is booming today!'}
Sent: {'timestamp': 1776452618.7254426, 'user': 'user_111', 'text': 'Bitcoin is falling badly'}
Sent: {'timestamp': 1776452618.826117, 'user': 'user_849', 'text': 'I hate this market crash'}
^C
(base) mugulinux@MUGUNDHAN:/mnt/d/MLDB_project$ tail -f spark.log
Inserted 2 records with latency
Inserted 2 records with latency
Inserted 2 records with latency
Inserted 2 records with latency
Inserted 3 records with latency
Inserted 2 records with latency
Inserted 3 records with latency
Inserted 2 records with latency
Inserted 2 records with latency
Inserted 2 records with latency
Inserted 2 records with latency
Inserted 2 records with latency
Inserted 3 records with latency
Inserted 4 records with latency
Inserted 2 records with latency
Inserted 2 records with latency
Inserted 2 records with latency
Inserted 2 records with latency
Inserted 3 records with latency
Inserted 2 records with latency
Inserted 2 records with latency
Inserted 3 records with latency
Inserted 2 records with latency
Inserted 4 records with latency
```

Step 4: Storage in MongoDB

Each processed record contains:

- Text
- Sentiment label
- Sentiment score
- Model predictions
- Latency values

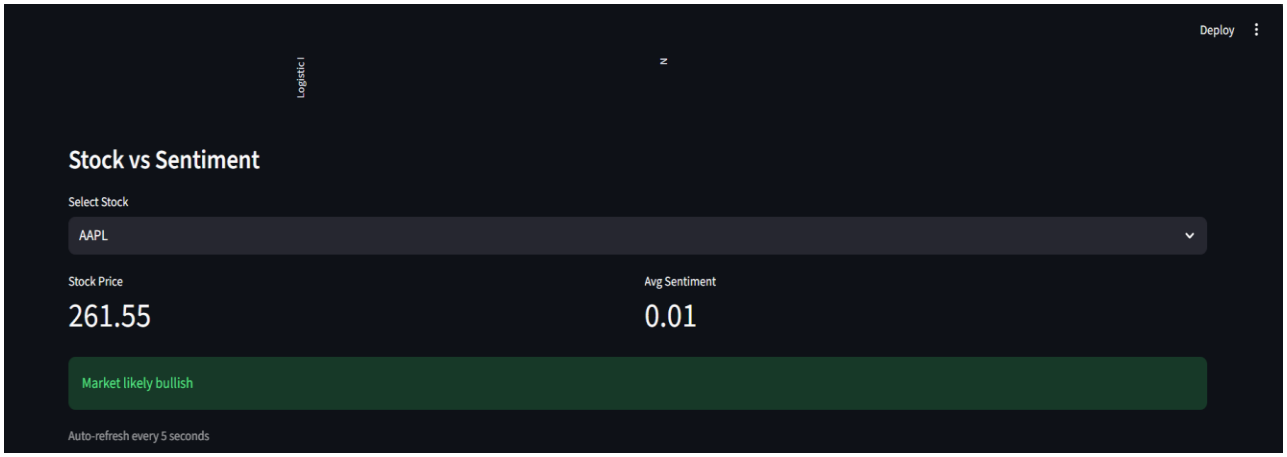
Step 5: Visualization using Streamlit

The dashboard retrieves data and displays:

- Sentiment distribution
- Latest tweets
- Model latency comparison

	user	text	sentiment	score
0	user_948	Market uncertainty is rising	negative	-0.34
1	user_239	I hate this market crash	negative	-0.7506
2	user_148	I hate this market crash	negative	-0.7506
3	user_989	Amazing growth in tech stocks	positive	0.7506
4	user_971	I hate this market crash	negative	-0.7506
5	user_367	This is the worst investment ever	negative	-0.6249
6	user_847	Tesla stock looks promising	positive	0.4019
7	user_643	Market uncertainty is rising	negative	-0.34
8	user_894	Tesla stock looks promising	positive	0.4019
9	user_364	This is the worst investment ever	negative	-0.6249

	user	text	sentiment	score
4	user_971	I hate this market crash	negative	-0.7506
5	user_367	This is the worst investment ever	negative	-0.6249
6	user_847	Tesla stock looks promising	positive	0.4019
7	user_643	Market uncertainty is rising	negative	-0.34
8	user_894	Tesla stock looks promising	positive	0.4019
9	user_364	This is the worst investment ever	negative	-0.6249



6. Models Used and Their Working

The project uses three different approaches for sentiment analysis:

1. VADER (Rule-Based Model)

VADER (Valence Aware Dictionary and Sentiment Reasoner) is a lexicon-based model designed specifically for social media text.

- Uses predefined sentiment scores for words
- Calculates a compound score
- Classifies sentiment based on thresholds

2. Naive Bayes Model

Naive Bayes is a probabilistic classifier based on Bayes' theorem.

Working:

- Converts text into numerical features using:
    - Tokenization
    - HashingTF
    - IDF
- Computes probability of each class
- Assigns the class with highest probability

3. Logistic Regression

Logistic Regression is a supervised learning model used for classification.

Working:

- Uses the same feature extraction pipeline
- Applies a linear decision boundary
- Outputs probability scores

7. Evaluation of Models

The performance of the models was evaluated using standard classification metrics.

Results:

Accuracy : 0.6988

Precision : 0.8046

Recall : 0.6988

F1 Score : 0.6407

Confusion Matrix:

```
[[55686      0      0]
 [37362  18563  18651]
 [    0      0  55730]]
```

Interpretation

- The model achieves **~70% accuracy**, which is reasonable for real-time sentiment analysis
- Precision is high (0.80), meaning predictions are reliable
- Recall is moderate, indicating some misclassification
- F1 score balances both precision and recall

The confusion matrix shows that:

- Positive and neutral classes are predicted well
- Some confusion exists in mixed sentiment cases

8. Observations

During experimentation, the following observations were made:

- VADER is **fast but less accurate**
- Naive Bayes is **efficient but limited**
- Logistic Regression provides **better balance between accuracy and latency**

9. Project Deliverables

This project improves upon existing solutions by:

1. Implementing a **complete real-time pipeline**
2. Integrating **multiple ML models simultaneously**
3. Comparing **latency across models**
4. Using **distributed streaming architecture**
5. Providing **live visualization dashboard**

10. Conclusion

This project successfully demonstrates the design and implementation of a **real-time sentiment analysis system** using modern big data tools.

The system is capable of:

- Streaming data in real time
- Processing it using multiple ML models
- Storing results efficiently
- Visualizing insights dynamically

Contribution Parts

- Mugundhan B (M25DE1037) - spark/, kafka/, run.sh
- Kakarla Sai Swaroop (M25DE1023) - ml/, evaluation/
- Abhinav Tote (M25DE1041) – stock/, dashboard/